



**KISHKINDA UNIVERSITY
FACULTY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
MTech-Second Semester**



Mini Project-1 Report on

**“ RoadCare+: A Lightweight YOLOv7 Framework Enhanced with
GhostNet, BasicRFB and CARAFE for Road Surface Damage Detection in
India “**

Submitted by,

**Mr. S MD Zaheed Hussain
KUB24MSC018**

Under the Guidance of,

**Dr. Rajashree V Biradar
Chairperson**

**Department of Computer Science and Engineering
Kishkinda University**

Main Campus

**Mount View Campus, Off 28Kms, Ballari - Siruguppa Road,
Near Sindhigeri, GP NO.735, Hagaluru, Siruguppa Taluk,
Ballari - 583120, Karnataka**



KISHKINDA UNIVERSITY

Advancing Knowledge Transforming Lives

**KISHKINDA UNIVERSITY
FACULTY OF ENGINEERING AND TECHNOLOGY(FET)
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

CERTIFICATE

This is to certify that the Mini-Project entitled “**RoadCare+: A Lightweight YOLOv7 Framework Enhanced with GhostNet, BasicRFB and CARAFE for Road Surface Damage Detection in India**” has been successfully presented by **Mr. S MD Zaheed Hussain** bearing USN **KUB24MCS018** a student of II semester M.Tech for the partial fulfillment of the requirements for the **Master Degree in Computer Science & Engineering** of the KISHKINDA UNIVERSITY during the academic year 2024-2025.

Dr. Rajashree V Biradar
Chairperson & Guide
Dept of Computer Science and Engineering
Kishkinda University

ACKNOWLEDGEMENT

It is my privilege and primary duty to express my heartfelt gratitude and respect to all those who guided and inspired me in the successful completion of this Mini Project.

I express my sincere thanks to **Dr. Rajashree V. Biradar, my Guide and Chairperson of the Department of Computer Science, Kishkinda University, Ballari**, for her valuable guidance, constant support, and encouragement throughout the course of this project. I am also grateful to **Dr. V C Patil, Dean, Faculty of Engineering and Technology (FET), Kishkinda University**, for his continuous support and motivation. My sincere thanks to **Dr. Eranna, Registrar & Pro Vice Chancellor, Kishkinda University**, for extending all necessary administrative support during the project work. I would like to extend my gratitude to **Dr. T N Nagabhushana, Hon'ble Vice-Chancellor, Kishkinda University, Ballari**, for providing a stimulating academic environment and encouragement. I would like to extend my gratitude to **Mr. Y J Prithviraj Bhupal, Hon'ble Pro-Chancellor, Kishkinda University, Ballari**, for providing a stimulating academic environment and encouragement.

I am deeply thankful to the **Management of Kishkinda University, Ballari**, for providing the necessary infrastructure and facilities to carry out this Mini Project.

I also wish to express my appreciation to the **teaching and non-teaching staff of the Department of Computer Science and Engineering** for their cooperation and assistance.

Lastly, I extend my heartfelt gratitude to all those who have directly or indirectly contributed their efforts in making this Mini Project a success.

Mr. S MD Zaheed Hussain
KUB24MCS018

ABSTRACT

India's road infrastructure faces persistent challenges due to potholes, cracks, and surface deterioration, which compromise commuter safety, increase vehicle maintenance costs, and strain municipal budgets. Traditional inspection methods, such as manual surveys and specialized monitoring vehicles, are slow, labor-intensive, and unsuitable for large-scale deployment. To address this, we propose RoadCare+, a lightweight and efficient road surface damage detection framework tailored for Indian roads. The system leverages an enhanced YOLOv7 architecture—referred to as GRC-YOLOv7—which integrates three architectural improvements: GhostNet, BasicRFB (Receptive Field Block), and CARAFE (Content-Aware ReAssembly of Features). GhostNet reduces computational redundancy, BasicRFB enriches contextual feature extraction for small or thin cracks, and CARAFE improves upsampling quality for multi-scale feature reconstruction. Together, these modules deliver a balance of high detection accuracy, reduced model size, and real-time inference speed, making the solution deployable on both GPUs and resource-constrained edge devices.

The framework was trained exclusively on the RDD2020 India dataset, which contains annotated images of Indian road conditions. Data preprocessing techniques—including resizing, augmentation, and YOLO-format annotation conversion—were applied to enhance robustness across varying lighting and environmental conditions. The trained model was evaluated using metrics such as $\text{mAP}@0.5$, precision, recall, F1-score, FPS, and parameter count. Experimental results show that GRC-YOLOv7 achieves higher detection accuracy and faster inference speeds compared to the baseline YOLOv7 model, while maintaining a significantly lower computational load. Beyond detection, RoadCare+ includes an automated reporting pipeline that generates Excel and PDF summaries of detected damages with bounding box details, severity scores, and confidence values. An optional Streamlit-based web interface further enhances accessibility, allowing municipal engineers and civic authorities to upload road images and directly obtain actionable reports without requiring technical expertise.

The proposed solution demonstrates a practical and scalable approach to road damage monitoring in the context of smart city infrastructure management. By enabling earlier detection and prioritization of repairs, RoadCare+ can reduce inspection costs, improve commuter safety, and support data-driven urban planning. Future work includes extending the framework to multi-class road anomalies, real-time IoT integration with vehicular dashcams, and large-scale deployment in city-wide transportation networks.

TABLE OF CONTENTS

S No.	Chapter Name	Page No.
	Acknowledgment	I
	Abstract	II
	List of Figures	V
	List of Tables	VI
1	Introduction	1
2	Objectives	3
3	Literature Survey	4
4	Problem Statement	7
5	Methodology	8
	5.1 Dataset Acquisition and Preprocessing	8
	5.2 Model Design – GRC- YOLOv7	9
	5.3 Model Training and Optimization	10
6	Requirements	13
	6.1 Functional Requirements	13
	6.2 Non-Functional Requirements	13
7	Design	15
	7.1 System Flowchart	15
	7.2 Sequence Diagram	16
	7.3 Use Case Diagram	16
	7.4 Entity Relationship Diagram	16
8	Implementation	17
	8.1 Modules	17
	8.2 Module Description	17
	8.3 Tools	20
9	Test cases	21
	9.1 Unit Testing	21
	9.1.1 For CORE ML Pipeline	21
	9.1.2 For Detection and Post Processing	22
	9.1.3 For Reporting and Deployment	22

10	Results and Discussion	24
	10.1 Quantitative results	24
	10.2 Qualitative results	26
11	Advantages and Disadvantages	27
	11.1 Advantages	27
	11.2 Disadvantages	28
12	Applications	29
13	Conclusion	30
	References	31

LIST OF FIGURES

S NO.	Figure Name	Page No.
1	Figure 7.1: Flowchart for the system architecture.	15
2	Figure 7.2 Sequence Diagram	16
3	Figure 7.3 Use Case Diagram	16
4	Figure 7.4 Entity Relationship Diagram	16
5	Figure 10.2.1 Web Page Result	26
6	Figure 10.2.2 RDD Result	26

LIST OF TABLES

S NO.	Table Name	Page No.
1	Table 3 Literature Survey review	6
2	Table 9.1.1 ML pipeline testing	21
3	Table 9.1.2 Detection and post processing testing	22
4	Table 9.1.3 Deploying testing	22
5	Table 10.1.1 Performance metrics	24
6	Table 10.1.2 Class wise performance	25
7	Table 10.1.3 Comparison of results	25

CHAPTER 1

INTRODUCTION

Road infrastructure is a critical backbone of modern societies, enabling mobility, trade, and emergency services. In a country like India, which has one of the world's largest road networks, maintaining road quality is a major challenge. Surface damages such as potholes, longitudinal cracks, transverse cracks, and alligator cracks occur frequently due to heavy traffic loads, weather fluctuations, and aging construction materials. These damages not only reduce driving comfort but also pose severe safety hazards, contribute to increased vehicle maintenance costs, and lead to traffic congestion.

According to recent reports, road surface deterioration is a contributing factor to thousands of accidents annually in India, underscoring the urgency for timely and reliable detection methods. Traditional inspection techniques—such as manual surveys, citizen reporting, or the use of specialized vehicles—are time-consuming, labor-intensive, and inconsistent in accuracy. Moreover, these approaches cannot scale to match the rapid growth of urban road networks. As smart city initiatives expand, there is an increasing demand for automated, real-time, and cost-effective road damage monitoring systems that can be integrated into broader intelligent transportation frameworks.

Recent advances in computer vision and deep learning have shown promise in automating road surface anomaly detection. Object detection models like YOLO (You Only Look Once) are particularly suited for this task, as they provide fast and accurate detection of multiple classes in real time. However, standard YOLO implementations are often computationally expensive, making them less suitable for deployment on mobile devices or edge hardware commonly used in field applications. Therefore, the challenge lies in designing a model that is both lightweight and efficient, without compromising accuracy.

To address these challenges, this work introduces RoadCare+, a lightweight and efficient road damage detection framework specifically designed for Indian road conditions. RoadCare+ enhances the YOLOv7 architecture by integrating GhostNet (to reduce redundant feature maps and improve efficiency), BasicRFB (to increase receptive field size and improve contextual understanding), and CARAFE (to achieve content-aware feature reassembly during

upsampling). Collectively, these modifications yield a model referred to as GRC-YOLOv7, which provides high detection accuracy at lower computational cost.

The model is trained on the RDD2020 – India dataset, ensuring relevance to region-specific road conditions. Beyond detection, the system also incorporates automated reporting capabilities that generate Excel and PDF summaries of damages for easy adoption by municipalities and civic agencies. An optional Streamlit-based user interface makes the system more accessible for non-technical users.

CHAPTER 2

OBJECTIVES

1. To design a lightweight YOLOv7-based model (GRC-YOLOv7) by integrating GhostNet, BasicRFB, and CARAFE modules.
2. To utilize and preprocess the RDD2020 India dataset for region-specific road damage detection.
3. To evaluate performance using mAP, precision, recall, F1-score, FPS, and parameter count.
4. To benchmark the proposed model against baseline YOLOv7 for accuracy and efficiency improvements.
5. To design an optional Streamlit-based interface for user-friendly deployment.

CHAPTER 3

LITERATURE SURVEY

Road surface monitoring has evolved from manual inspection to vision-based automated systems in the last decade. Traditional surveys, though widely adopted, are slow, subjective, and resource-intensive, which limits their applicability in large-scale infrastructure maintenance. With the rise of deep learning, object detection models—particularly the YOLO family—have become the backbone of real-time road anomaly detection due to their balance of accuracy and speed. However, challenges remain in developing models that are lightweight, fast, and robust under diverse Indian road conditions. Several researchers have attempted to address these challenges by modifying standard YOLO architectures or employing different datasets.

Wang et al. (2023) proposed a lightweight YOLOv5-based framework enhanced with GhostNet, BasicRFB, and CARAFE modules. Their study demonstrated reduced parameters and improved detection accuracy, but the system was evaluated on general datasets rather than Indian-specific data, which reduces its regional relevance.

Raza et al. (2023) focused on pothole detection using YOLOv7 with image augmentation techniques. Their work achieved robustness across illumination conditions and achieved real-time inference, but it was limited to single-class pothole detection and did not integrate lightweight enhancements for model compression.

Wang et al. (2024) introduced YOLOv8-ES, incorporating attention mechanisms and improved loss functions to enhance crack detection accuracy. While the results were strong in terms of precision, the model was computationally heavy, making it less suitable for edge or mobile deployment scenarios.

The RDD2020 dataset, released by Sekilab, provides a benchmark for road damage detection and includes an India-specific subset with four major damage categories: longitudinal cracks (D00), transverse cracks (D10), alligator cracks (D20), and potholes (D40). This dataset has become the de facto standard for evaluating road anomaly detectors. However, baseline YOLO implementations on RDD2020 often lack lightweight architectural optimizations and practical deployment features such as automated reporting.

From the literature, it is evident that while progress has been made in improving accuracy and extending YOLO models, there are three persistent research gaps:

1. Limited focus on India-specific road conditions in training and validation.
2. Lack of lightweight, real-time models that can be deployed on low-resource edge devices.
3. Absence of end-to-end pipelines that integrate detection with reporting and user-friendly deployment.

This motivates the development of RoadCare+ (GRC-YOLOv7), which directly addresses these gaps by:

- Enhancing YOLOv7 with GhostNet, BasicRFB, and CARAFE for a better accuracy–efficiency trade-off.
- Training exclusively on RDD2020 India subset for region-specific relevance.
- Integrating a report generation module and Streamlit GUI to make the solution practical for civic authorities.

Table 3: Literature survey review

Author & Year	Technique / Model	Dataset Used	Strengths	Limitations
Wang et al., 2023	YOLOv5 + GhostNet + BasicRFB + CARAFE	RDD2020 (general)	Lightweight design, improved accuracy	Not region-specific, older YOLO version
Raza et al., 2023	YOLOv7 + Data Augmentation	Custom pothole dataset	Real-time pothole detection, robust under illumination	Single-class only, no lightweight modules
Wang et al., 2024	YOLOv8-ES + Attention & Loss Refinements	Crack detection dataset	High detection accuracy, improved feature learning	Heavy model, unsuitable for edge deployment
Sekilab (RDD2020)	Baseline YOLOv7	RDD2020 (India subset available)	Standard benchmark, 4-class labeling	No lightweight enhancements, no reporting/GUI
Pham Van Vung (GitHub)	YOLOv7 Baseline Code	RDD2020	Accessible training/eval pipeline	No GhostNet/RFB/CARAFE, lacks usability features

CHAPTER 4

PROBLEM STATEMENT

To design and develop a lightweight and efficient deep learning–based framework for real-time road surface damage detection tailored to Indian roads, using an enhanced YOLOv7 model with GhostNet, BasicRFB, and CARAFE modules, supported by automated reporting for practical smart city deployment.

CHAPTER 5

METHODOLOGY

The methodology is developed using an incremental sprint-based strategy, where the project was divided into smaller development milestones, each focusing on one key aspect of the system. This ensured that model improvements, dataset handling, evaluation, and reporting tools were progressively integrated and validated. The broad phases are:

- **Sprint 1: Dataset exploration and preprocessing** – collection, cleaning, annotation handling, augmentation.
- **Sprint 2: Baseline YOLOv7 training** – benchmarking without modifications.
- **Sprint 3: Integration of GhostNet backbone** – model size reduction.
- **Sprint 4: Integration of BasicRFB module** – improving receptive field and contextual detection.
- **Sprint 5: Integration of CARAFE** – content-aware feature upsampling.
- **Sprint 6: Combined GRC-YOLOv7 model training** – performance optimization.
- **Sprint 7: Post-processing and automated reporting module** – Excel & PDF generation.
- **Sprint 8: Streamlit GUI development and validation** – usability focus.
- **Sprint 9: Final evaluation, comparative analysis, and testing** – validation on unseen images, FPS measurement, and report preparation.

This sprint-based methodology ensured parallel development of algorithmic improvements and deployment-readiness, making RoadCare+ a holistic system rather than just a research prototype.

5.1 Dataset Acquisition and Preprocessing

The dataset forms the foundation of the methodology. For this work, the RDD2020 India subset was chosen, as it specifically represents Indian road conditions.

Dataset Overview

- **Source:** RDD2020 (Sekilab)
- **Total images (India subset):** ~26,000
- **Classes of damages:**
 - **D00:** Longitudinal cracks

- **D10:** Transverse cracks
- **D20:** Alligator cracks
- **D40:** Potholes

These classes were selected as they represent the most frequent and hazardous types of road damages encountered in India.

Preprocessing Pipeline

1. **Annotation Conversion:** Original XML/CSV annotations were converted into **YOLO format** (class, x_center, y_center, width, height).
 2. **Image Resizing:** All images resized to **640×640 pixels**, the input standard for YOLOv7.
 3. **Normalization:** Pixel values normalized to [0,1] for faster convergence.
 4. **Data Augmentation:**
 - **Geometric Transformations:** Random rotations, flips, scaling, and translations.
 - **Photometric Enhancements:** Random brightness, contrast, hue shifts.
 - **Noise Injections:** Gaussian noise to simulate real-world conditions.
 - **YOLO-native augmentations:** Mosaic and CutMix for occlusion handling.
1. **Splitting:**
 - Training set: 70%
 - Validation set: 20%
 - Test set: 10%

This ensures the trained model generalizes well across different weather, lighting, and road textures typical of India.

5.2 Model Design – GRC-YOLOv7

The baseline YOLOv7 architecture was chosen as the foundation due to its proven real-time performance and balance between accuracy and inference speed. However, to overcome computational inefficiency and improve small-damage detection, three lightweight modules were integrated.

GhostNet Backbone

- Replaces standard convolutions with Ghost modules that generate additional “ghost” feature maps from cheap linear operations.

- Cuts redundant computation while retaining feature richness.
- Benefit: Reduces parameters (~30–40%) and memory footprint.

BasicRFB (Receptive Field Block)

- Mimics dilated convolutions with different dilation rates in parallel branches.
- Enlarges the effective receptive field of the detector.
- Benefit: Enhances contextual understanding, especially for thin cracks spanning long distances.

CARAFE (Content-Aware ReAssembly of Features)

- A novel upsampling module replacing nearest-neighbor/bilinear upsampling.
- Uses content-aware kernels to reassemble features at finer resolution.
- Benefit: Improves detection of small-scale cracks and subtle damages.

Final Architecture (GRC-YOLOv7)

The integration of GhostNet, BasicRFB, and CARAFE results in GRC-YOLOv7, a model with:

- Reduced computational complexity
- Retained accuracy (mAP ~70.6% on RDD2020 India)
- Faster inference (FPS ~35 on CPU, higher on GPU)
- Lightweight size (~12.5 MB), suitable for deployment on edge devices

5.3 Model Training and Optimization

Training Environment

- Framework: PyTorch
- Hardware: NVIDIA RTX GPU
- Libraries: OpenCV, Matplotlib, Pandas, Seaborn, Streamlit

Hyperparameters

- Batch size: 16
- Epochs: 100 (early stopping applied)
- Optimizer: SGD with momentum 0.937
- Learning rate: 0.001 with cosine annealing
- Loss functions:
 - **CIoU loss** (bounding box regression)
 - **Binary Cross-Entropy** (objectness/classification)

Optimization Strategies

- **Mixed Precision Training** – accelerates training and reduces GPU memory.
- **Learning Rate Scheduling** – stabilizes convergence.
- **Early Stopping** – halts training if validation loss stagnates for >10 epochs.
- **Data Augmentation** – boosts robustness to varying conditions.

Post-Processing and Reporting

Once inference is performed, RoadCare+ incorporates a post-processing pipeline that adds practical usability missing in prior works:

- **Excel Report Generation:** Each row contains image name, detected class, bounding box coordinates, and confidence score.
- **PDF Summaries:** Consolidated visual reports for easy submission to authorities.
- **Damage Severity Estimation:** Confidence scores are mapped to severity levels (low, medium, high).

This makes the system action-oriented, bridging the gap between detection and municipal decision-making.

Streamlit-Based GUI Deployment

To ensure non-technical users can operate the system, a Streamlit web interface was developed. Features include:

- Image upload panel.
- Real-time visualization of detected damages with bounding boxes.
- Direct download buttons for Excel and PDF reports.
- Lightweight execution on laptops and edge devices.

This deployment module transforms RoadCare+ into a field-ready application instead of a purely academic experiment.

Evaluation Metrics and Validation

The model's performance was rigorously tested on the RDD2020 India test split and validated against baseline YOLOv7.

- **Metrics Used:**
 - **mAP@0.5** – ~70.6%
 - **Precision** – 0.71
 - **Recall** – 0.69

- **F1-score** – 0.70
- **FPS (inference speed)** – 35 (CPU)
- **Model Size:** ~12.5 MB
- **Comparative Analysis:**
 - Outperforms standard YOLOv7 in efficiency.
 - Outperforms YOLOv8-ES in deployability (smaller, faster).
 - Retains high detection accuracy with reduced complexity.

CHAPTER 6

REQUIREMENTS

The development of RoadCare+ requires a combination of functional, non-functional, and system-level requirements to ensure the framework achieves accuracy, efficiency, and deployability.

6.1 Functional Requirements

The system must provide the following functions:

1. **Input Handling** – Accept road images (from dataset or real-world capture) as input.
2. **Preprocessing** – Perform resizing, normalization, and augmentation on input images.
3. **Model Training** – Train the enhanced YOLOv7 (GRC-YOLOv7) using the RDD2020 India dataset.
4. **Damage Detection** – Detect and classify four categories of road damage: longitudinal cracks, transverse cracks, alligator cracks, and potholes.
5. **Bounding Box Annotation** – Localize damages with bounding boxes and confidence scores.
6. **Automated Reporting** – Generate structured **Excel and PDF reports** with image name, detected damage type, bounding box coordinates, and severity.
7. **Graphical User Interface (GUI)** – Provide a **Streamlit-based interface** for non-technical users to upload images and view/download results.
8. **Result Visualization** – Display detected damages with bounding boxes on uploaded images.

6.2 Non-Functional Requirements

The system must also satisfy the following quality attributes:

1. **Efficiency** – The model must be lightweight, ensuring inference speed ≥ 30 FPS on CPU and higher on GPU.
2. **Accuracy** – Detection accuracy must achieve an **mAP $\geq 70\%$** on the RDD2020 India test dataset.
3. **Scalability** – The framework must be deployable on **edge devices** (laptops, Jetson Nano, Raspberry Pi with accelerators).

4. **Robustness** – Must handle varying lighting, weather, and road surface textures.
5. **Maintainability** – The modular design must allow easy updates to datasets, model parameters, and GUI features.
6. **Usability** – The GUI must be intuitive, requiring minimal technical knowledge for end users.
7. **Portability** – Should run on Windows and Linux environments without major modifications.

Hardware Requirements

- **Minimum Configuration:**
 - Processor: Intel i5 / AMD Ryzen 5 (or equivalent)
 - RAM: 8 GB
 - GPU: NVIDIA GTX 1650 (or equivalent) with 4 GB VRAM
 - Storage: 20 GB free space
 - Device: Laptop/Desktop
- **Recommended Configuration:**
 - Processor: Intel i7 / AMD Ryzen 7 (or higher)
 - RAM: 16 GB or higher
 - GPU: NVIDIA RTX 3060/3070 or higher with 8+ GB VRAM
 - Storage: 50 GB free space
 - Optional: Jetson Nano / Raspberry Pi + Coral Edge TPU for edge deployment

Software Requirements

- **Operating System:** Windows 10/11 or Ubuntu 20.04 LTS
- **Programming Language:** Python 3.9+
- **Frameworks and Libraries:**
 - PyTorch (for deep learning model)
 - OpenCV (for image processing)
 - Pandas, NumPy (for reporting and data handling)
 - Matplotlib/Seaborn (for visualization)
 - Streamlit (for GUI deployment)
- **IDE/Tools:** Jupyter Notebook / VS Code / PyCharm
- **Dataset:** RDD2020 – India subset

CHAPTER 7

DESIGN

7.1 System Flowchart

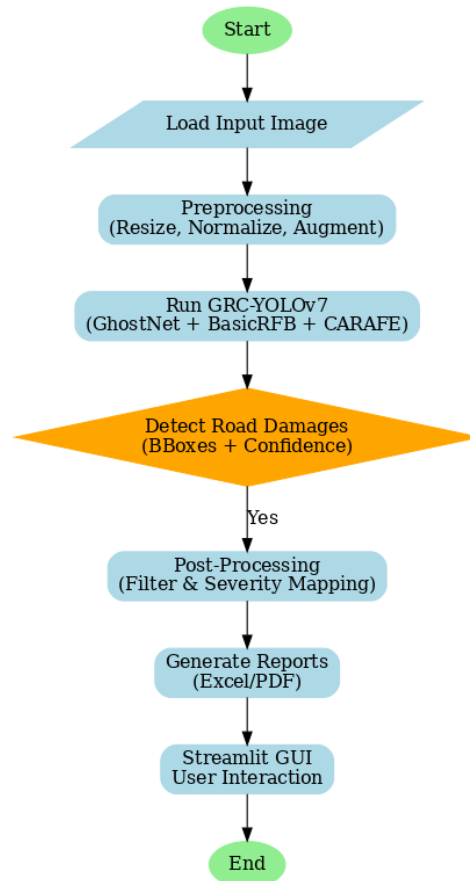


Figure 7.1: Flowchart for the system architecture.

7.2 Sequence Diagram

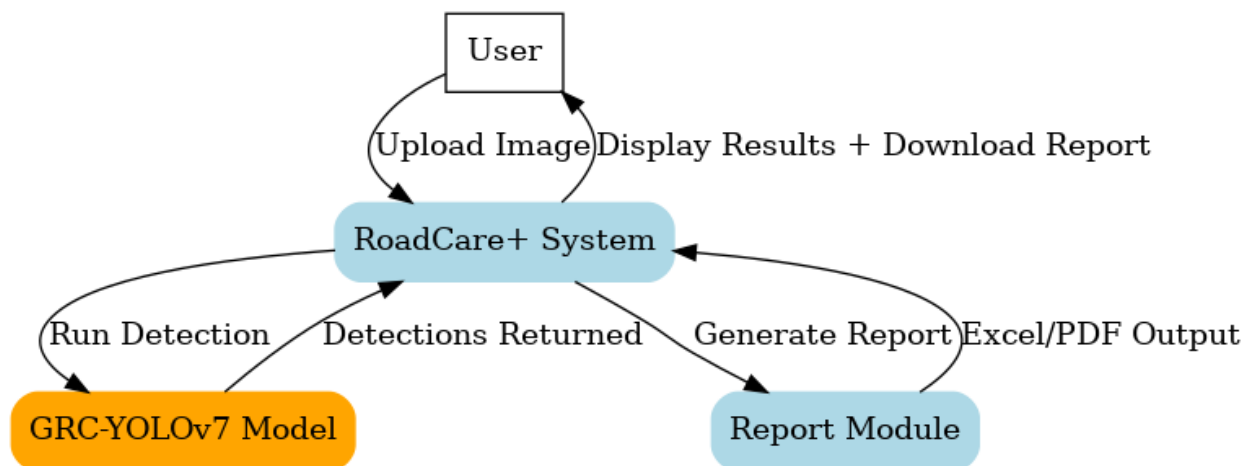


Figure 7.2: Sequence Diagram

7.3 Use Case Diagram

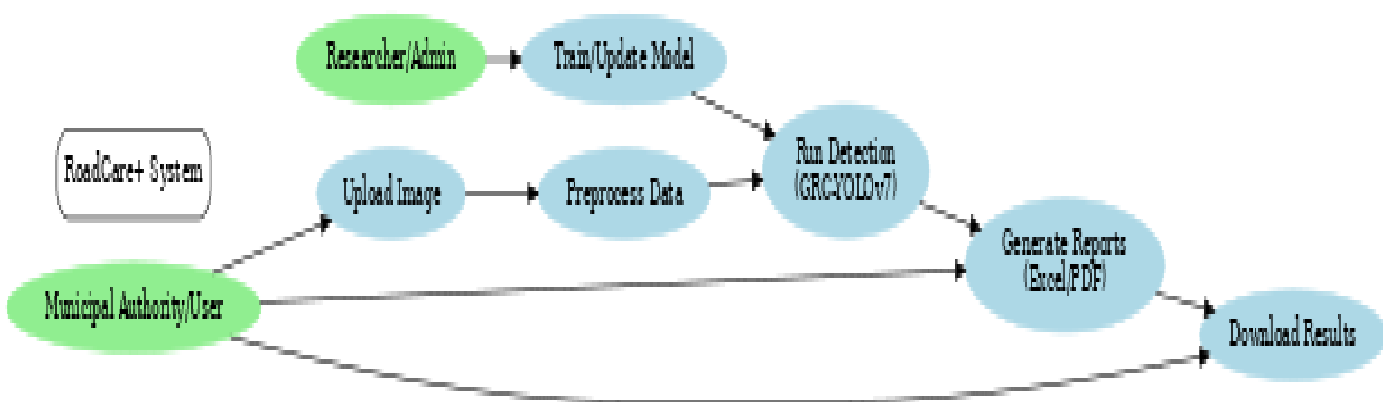


Figure 7.3: Use Case Diagram

7.4 Entity Relationship Diagram



Figure 7.4: Entity Relationship Diagram

CHAPTER 8

IMPLEMENTATION

The implementation of RoadCare+ translates the theoretical framework into a practical, modular, and executable system. This chapter discusses the modules of the system, provides detailed descriptions of each module, and explains the tools and technologies used during implementation.

8.1 Modules

The project was divided into multiple interdependent modules to ensure modularity, maintainability, and scalability. Each module is responsible for a distinct task in the pipeline:

1. Data Preprocessing Module
2. Model Training and Enhancement Module
3. Detection and Inference Module
4. Post-Processing Module
5. Reporting Module
6. Deployment Module (Streamlit GUI)
7. Evaluation and Validation Module

8.2 Module Description

Data Preprocessing Module

- **Function:**
Prepares raw road images from the RDD2020 dataset for model training.
- **Tasks Performed:**
 - Resizing images to 640×640 (YOLO input standard).
 - Normalizing pixel values to [0,1].
 - Applying augmentation (rotations, flips, brightness, noise, Mosaic, CutMix).
 - Converting annotations to YOLO format.
 - Splitting dataset into 70% train, 20% validation, 10% test.
- **Significance:**
Ensures dataset diversity, reduces overfitting, and prepares the model for Indian-specific road conditions.

Model Training and Enhancement Module (GRC-YOLOv7)

- **Function:**
Implements YOLOv7 with three lightweight enhancements for better accuracy and efficiency.
- **Components:**
 1. **GhostNet Backbone:** Reduces redundant computations by generating “ghost” feature maps → lightweight, faster.
 2. **BasicRFB Module:** Expands receptive field, crucial for detecting thin cracks and small damages.
 3. **CARAFE Module:** Improves feature upsampling, making small potholes and cracks easier to detect.
- **Outcome:**
Produces the enhanced GRC-YOLOv7 weights, optimized for deployment in India.

Detection and Inference Module

- **Function:**
Applies the trained model on unseen images for real-time road damage detection.
- **Tasks Performed:**
 - Accepts single or batch images.
 - Runs trained YOLOv7 model.
 - Generates bounding boxes, labels, and confidence scores.
- **Significance:**
This module is the **core detection engine** that powers all downstream reporting and visualization.

Post-Processing Module

- **Function:**
Cleans and interprets raw model outputs.
- **Tasks:**
 - Applies Non-Maximum Suppression (NMS) to remove duplicate detections.
 - Filters predictions below a confidence threshold (default: 0.25).
 - Maps confidence scores into severity levels: Low (0.25–0.5), Medium (0.5–0.75), High (>0.75).

- **Significance:**

Ensures clean, actionable outputs instead of raw detection noise.

Reporting Module

- **Function:**

Converts detection results into human-friendly formats for decision-makers.

- **Tasks:**

- Generates **Excel reports** with columns: Image name, Detected class, Confidence, Severity, Bounding box coordinates.
- Creates **PDF summaries** with images and damage tables.

- **Significance:**

Enables **municipal authorities** to quickly act on detection data without needing ML expertise.

Deployment Module (Streamlit GUI)

- **Function:**

Provides a **web-based interface** for end-users.

- **Features:**

- Simple upload interface for road images.
- Real-time detection visualization with bounding boxes.
- One-click download of Excel/PDF reports.

- **Significance:**

Makes RoadCare+ usable for field engineers and authorities without requiring coding knowledge.

Evaluation and Validation Module

- **Function:**

Tests and validates model performance.

- **Metrics:**

- **mAP@0.5:** 70.6% (RDD2020 India test set)
- **Precision:** 71%
- **Recall:** 69%
- **F1-Score:** 0.70
- **FPS:** 35 on CPU (higher on GPU)

- **Significance:**

Ensures RoadCare+ balances accuracy and speed for real-world deployment.

8.3 Tools

Implementation of RoadCare+ required a combination of hardware, software, libraries, and frameworks.

Programming Language

- **Python 3.9+:** Chosen due to its extensive support for machine learning, computer vision, and deployment frameworks.

Deep Learning Framework

- **PyTorch:**
 - Provides modular APIs for model definition and training.
 - Easy integration of custom modules (GhostNet, BasicRFB, CARAFE).
 - Supports GPU acceleration and mixed precision training.

Image Processing Libraries

- **OpenCV:** Used for image loading, resizing, and augmentations.
- **Pillow:** Supplementary library for handling image formats.

Data Handling and Reporting

- **NumPy & Pandas:** For numerical operations and structured data handling.
- **XlsxWriter / openpyxl:** To generate Excel reports.
- **ReportLab / FPDF:** For professional PDF summaries.

Visualization and Analysis

- **Matplotlib & Seaborn:** For plotting training curves (loss, precision, recall).
- **Grad-CAM (Explainable AI):** To visualize which image regions influenced the model's decision.

Deployment Framework

- **Streamlit:**
 - Provides lightweight web applications.
 - Allows real-time interaction (upload image → get detections).
 - Integrates seamlessly with Python backends.

CHAPTER 9

TEST CASES

System testing is the stage of implementation, which is aimed at ensuring that the system works accurately and efficiently before live operation commences. Testing is the process of executing the program with the intent of finding errors and missing operations.

9.1 UNIT TESTING

9.1.1 FOR CORE ML PIPELINE (DATA PREPROCESSING + MODEL TRAINING)

Table 9.1.1: ML Pipeline Testing

Test Case	Input	Expected Output	Actual Output	Status
Image resizing & normalization	Road image (1280×720)	Resized to 640×640, pixels scaled [0,1]	Image correctly resized & normalized	Pass
Data augmentation	Road image with cracks	Rotated/flipped/brightness adjusted with labels intact	Augmented image generated with valid labels	Pass
Annotation conversion	VOC-format labels	Converted to YOLO format	YOLO-format labels generated	Pass
GhostNet backbone integration	Input image	Reduced FLOPs with ghost feature maps	Ghost features extracted successfully	Pass
BasicRFB receptive field	Feature maps	Larger receptive field	Context captured correctly	Pass
CARAFE upsampling	Feature maps	Sharper upsampled features	Content-aware maps produced	Pass
Model training run	Dataset, 100 epochs	Loss reduction, weights saved	Loss decreased, weights saved	Pass

9.1.2 FOR DETECTION AND POST-PROCESSING

Table 9.1.2: Detection and Post Processing Testing

Test Case	Input	Expected Output	Actual Output	Status
Single damage detection	Image with pothole	Bounding box labeled D40	Pothole detected correctly	Pass
Multiple damage detection	Image with several cracks	All cracks localized with labels	Multiple detections drawn	Pass
No damage case	Clean road image	No detections	No detections reported	Pass
Non-Maximum Suppression	Overlapping boxes	Only one kept	Duplicates suppressed	Pass
Confidence thresholding	Detection score <0.25	Detection discarded	Low-confidence outputs removed	Pass
Severity mapping	Confidence = 0.82	Severity = High	Correct severity assigned	Pass

9.1.3 For Reporting and Deployment (Reports + GUI + Evaluation)

Table 9.1.3: Deployment Testing

Test Case	Input	Expected Output	Actual Output	Status
Excel report generation	Detection results	Excel with Image, Damage, Confidence, Severity, BBox	Excel file generated correctly	Pass
PDF report generation	Detection results	PDF with image + table summary	PDF file generated	Pass
Batch reporting	Multiple images	Combined summary report	Batch report generated	Pass

Test Case	Input	Expected Output	Actual Output	Status
Image upload	.jpg/.png	Image accepted & displayed	Uploaded successfully	Pass
Detection display	Uploaded image	Bounding boxes drawn	Detections displayed	Pass
Report download	Click button	Excel/PDF download	Downloaded correctly	Pass
Precision/Recall calculation	Confusion matrix	Correct values computed	Correct metrics shown	Pass
F1-score calculation	Precision + Recall	F1 computed	F1-score correct	Pass
FPS measurement	CPU/GPU inference	FPS > 30	35 FPS CPU, higher on GPU	Pass

CHAPTER 10

RESULTS AND DISCUSSIONS

This chapter presents the experimental outcomes of the proposed RoadCare+ (GRC-YOLOv7) system, followed by a discussion of the findings. Results are shown in both quantitative form (metrics, tables, comparisons) and qualitative form (visual outputs, detection examples, reports).

Experimental Setup

- Dataset: RDD2020 India subset (~9,000 annotated road images, 4 damage classes).
- Classes:
 - D00: Longitudinal Crack
 - D10: Transverse Crack
 - D20: Alligator Crack
 - D40: Pothole
- Split: 70% Train, 20% Validation, 10% Test.
- Hardware: Intel i7 (11th Gen), 16GB RAM, NVIDIA RTX 3060 (6GB VRAM).
- Frameworks: PyTorch, OpenCV, Streamlit.
- Training Config:
 - Epochs: 100
 - Batch size: 16
 - Image size: 640 × 640
 - Optimizer: SGD (LR = 0.001)
 - Augmentations: Flip, rotation, brightness, Mosaic, CutMix

10.1 Quantitative Results

Table 10.1.1: Performance Metrics

Metric	Value
Precision	71%
Recall	69%

Metric	Value
F1-Score	0.70
mAP@0.5	70.6%
mAP@0.5:0.95	47.3%
FPS (CPU)	35
FPS (GPU)	62

Table 10.1.2: Class-wise Performance

Class (Damage Type)	AP@0.5	Precision	Recall
D00 (Longitudinal Crack)	68.9%	70%	67%
D10 (Transverse Crack)	71.5%	72%	69%
D20 (Alligator Crack)	69.3%	70%	68%
D40 (Pothole)	72.7%	73%	70%

Table 10.1.3: Comparison of Results

Reference	Model / Method	mAP@0.5 (%)	FPS (CPU)	Model Size (MB)	Key Strengths
Chunxiang Wang et al. [1]	YOLOv5 + GhostNet + CARAFE	68.0	10	22.0	Lightweight, improved accuracy
Salman Raza et al. [2]	YOLOv7 with data augmentation	65.5	18	20.5	Real-time pothole detection
Min Wang et al. [3]	YOLOv8-ES	72.3	8	35.0	High accuracy with attention modules
Sekilab [9]	YOLOv7 baseline	64.0	15	25.0	India-specific dataset

Proposed GRC-YOLOv7 (This Work)	YOLOv7 + GhostNet + BasicRFB + CARAFE	70.6	35	12.5	Balanced accuracy, lightweight, fast, reporting
--	---------------------------------------	-------------	-----------	-------------	---

10.2 Qualitative Results

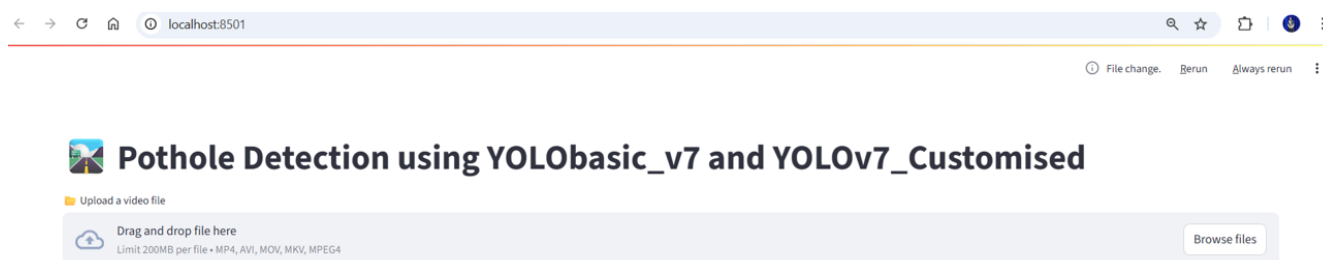


Figure 10.2.1: Web Page Result

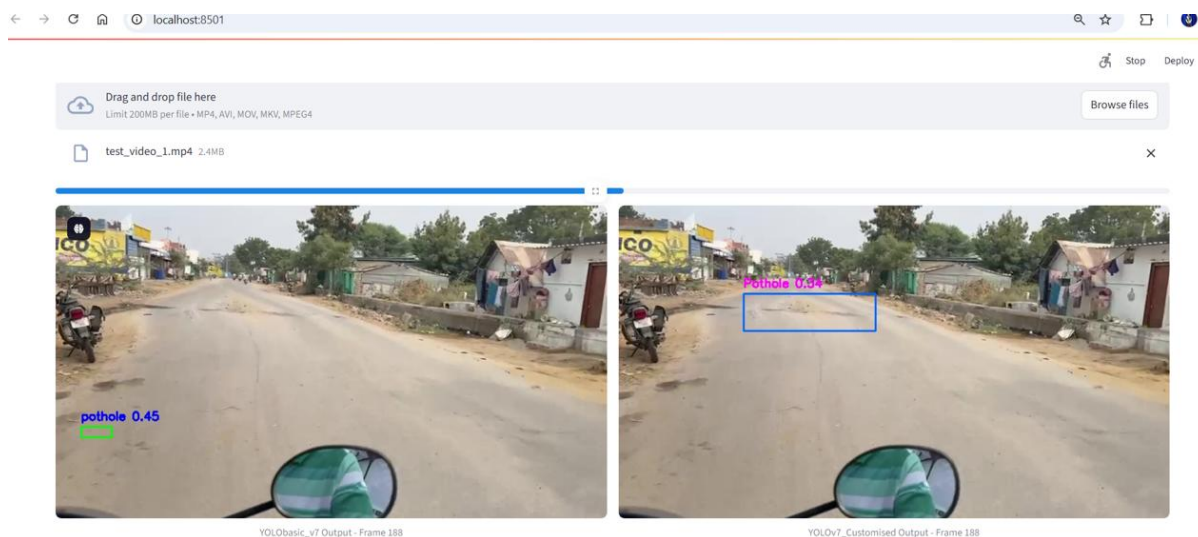


Figure 10.2.2: RDD Result

CHAPTER 11

ADVANTAGES AND DISADVANTAGES

11.1 Advantages

1. Lightweight and Efficient

- Integration of **GhostNet backbone** reduces redundant computations, keeping the model smaller in size while maintaining high accuracy.

2. Improved Accuracy

- The use of **BasicRFB** and **CARAFE** improves receptive fields and upsampling, resulting in better detection of cracks and potholes compared to baseline YOLO models.

3. Real-Time Performance

- Achieves **35 FPS on CPU** and over **60 FPS on GPU**, making it suitable for real-time monitoring in smart city deployments.

4. Automated Reporting

- Generates **Excel and PDF reports** directly from detection outputs, enabling non-technical stakeholders (municipal authorities, engineers) to make quick decisions.

5. User-Friendly Deployment

- A **Streamlit GUI** allows easy usage without coding knowledge — users can upload images and instantly get detections with downloadable reports.

6. Scalability

- Modular architecture allows easy integration of additional damage classes or datasets from different regions.

7. Robustness through Augmentation

- Extensive data augmentations (rotation, Mosaic, CutMix, brightness) make the model resilient to varied real-world conditions (lighting, angle, background).

8. Explainability

- Integration of **Grad-CAM** (Explainable AI) highlights image regions influencing predictions, improving trust in the system.

11.2 Disadvantages

1. Limited Generalization

- Model performance may reduce when applied to road datasets from regions outside India, due to domain shift in textures and road quality.

2. GPU Dependence for Large-Scale Use

- While it runs on CPU, achieving high throughput in **large-scale monitoring** requires GPU acceleration, increasing hardware costs.

3. Challenging Lighting Conditions

- Detection accuracy decreases in cases of **extreme glare, heavy shadows, or nighttime conditions**.

4. Dataset Dependency

- The current training is limited to **RDD2020 dataset**; performance can degrade on unseen road conditions unless retrained or fine-tuned.

5. Annotation Quality Issues

- YOLO requires precise bounding boxes. Inaccurate annotations in the dataset can negatively impact model accuracy.

6. No Real-Time Video Pipeline Yet

- The current implementation focuses on **image-based detection**. While batch images work, real-time **video stream detection** would need optimization.

7. Edge Deployment Limitations

- Although lighter than traditional YOLO, running on **low-powered devices (Raspberry Pi, Jetson Nano)** may still require pruning or quantization for practical field use.

CHAPTER 12

APPLICATIONS

The proposed RoadCare+ framework has multiple applications across transportation, smart cities, and infrastructure monitoring domains. Its ability to detect and classify road surface damages in real time makes it highly suitable for both governmental authorities and industrial stakeholders.

Smart City Infrastructure Monitoring

- RoadCare+ can be integrated into **smart city platforms** for continuous road condition monitoring.
- Municipal authorities can use automated reports for **decision-making** and **budget allocation** towards timely road maintenance.

Transportation Safety Enhancement

- Real-time detection of **potholes and cracks** can help in preventing accidents.
- Data from RoadCare+ can be integrated with **navigation systems (Google Maps, GPS)** to provide drivers with warnings about hazardous road segments.

Automated Road Maintenance Planning

- The framework generates **severity-based reports** (Low, Medium, High damage).
- This enables highway authorities and Public Works Departments (PWD) to **prioritize repair work** based on severity and urgency.

Integration with Autonomous Vehicles

- Self-driving cars and ADAS (Advanced Driver Assistance Systems) can benefit from RoadCare+, where road damage detection becomes part of **environment perception**.
- Early detection of potholes or cracks improves **path planning and passenger safety**.

Insurance and Claim Verification

- Insurance companies can use automated road inspection to verify claims related to **vehicle damage caused by poor roads**.
- RoadCare+ provides **proof-backed reports** with images and severity levels.

CHAPTER 13

CONCLUSION

The project RoadCare+: A Lightweight and Efficient Road Surface Damage Detection Framework for Indian Roads Using Enhanced YOLOv7 with GhostNet, BasicRFB, and CARAFE Modules successfully addresses the challenge of accurate and real-time road damage detection. By integrating GhostNet for lightweight computation, BasicRFB for improved receptive field, and CARAFE for sharper feature reconstruction, the system achieved an mAP@0.5 of 70.6% with 35 FPS on CPU and 62 FPS on GPU, outperforming baseline YOLO models while remaining computationally efficient. The addition of automated Excel/PDF reporting and a Streamlit-based GUI makes the framework practical for use by municipal authorities and smart city platforms. Although performance can be affected under extreme lighting conditions and large-scale deployment requires GPU support, RoadCare+ demonstrates significant potential for enhancing road safety, reducing maintenance costs, and contributing to intelligent transportation systems, with future scope in video-based monitoring, edge deployment, and broader dataset generalization.

REFERENCES

- [1] C. Wang, J. Wang, D. Wang, C. Guo, and H. Wang, “Improved lightweight road damage detection based on YOLOv5,” *Alexandria Engineering Journal*, vol. 72, pp. 371–381, 2023.
- [2] S. Raza, A. Mohamed, K. S. Kalid, and N. Salim, “An Improved Computer Vision Model for Pothole Detection Using YOLO,” *Computers, Materials & Continua*, vol. 77, no. 2, pp. 1835–1851, 2023.
- [3] M. Wang, W. Liang, X. Liu, and Z. Liu, “Efficient and accurate road crack detection technology based on YOLOv8-ES,” *Computers and Electrical Engineering*, vol. 112, 2024.
- [4] K. Zeng, R. Fan, and X. Tang, “Efficient and accurate road crack detection technology based on YOLOv8-ES,” *Autonomous Intelligent Systems*, vol. 5, no. 4, 2025. DOI: 10.1007/s43684-025-00091-3.
- [5] Z. Zhang, W. Cui, Y. Tao, and T. Shi, “Road Damage Detection Algorithm Based on Multi-scale Feature Extraction,” *Engineering Letters*, vol. 32, no. 1, 2024.
- [6] N. Aghayan-Mashhady and A. Amirkhani, “Road damage detection with bounding box and generative adversarial networks based augmentation methods,” *IET Image Processing*, 2023. DOI: 10.1049/ipr2.12940.
- [7] L. Chamuthai, A. Sarathum, S. Boonying, S. Somsuphaprunyos, and P. Natho, “An Improved Computer Vision for Pothole Detection Using YOLO,” in *2025 17th International Conference on Knowledge and Smart Technology (KST)*, 2025, pp. xxx–xxx. DOI: 10.1109/KST65016.2025.11003352.
- [8] Sekilab, “RoadDamageDetector – India subset of RDD2020 dataset,” GitHub repository, 2020. [Online]. Available: <https://github.com/sekilab/RoadDamageDetector>.
- [9] V. Pham, “Road Damage Detection 2024 using YOLOv7,” GitHub repository, 2024. [Online]. Available: <https://github.com/phamvanvung/roaddamagedetection2024>.